



Cours d'introduction au langage de programmation Python

Variables, types et opérateurs.

Une variable est un espace mémoire dans lequel il est possible de stocker une valeur (une donnée).

Les principaux types de variables en Python sont :

- **les entiers** : type int. Ils supportent les opérations +, -, *, /, ** (puissance), // (division entière), % (reste de la division entière), abs() (valeur absolue).

- **les booléens** sont des variables qui ne peuvent prendre que deux valeurs : False ou True. Ils sont générés par les opérateurs de comparaison <, >, <=, >=, == (test d'égalité), != (test de différence) et les opérateurs logiques not, or et and.

- **les nombres flottants** : type float. Ils supportent les opérations usuelles.

- **les chaînes** de caractères : type string.

- **les listes** : type list.

1- Le type int (integer : nombres entiers)

Pour affecter (on dit aussi assigner) la valeur 17 à la variable nommée Age :

```
>>> Age = 17
```

La fonction print affiche la valeur de la variable :

```
>>> print (Age)
```

```
17
```

La fonction type() retourne le type de la variable :

```
>>> print (type(Age))
```

```
<type 'int'>
```

int est le type des nombres entiers.

```
>>> # ceci est un commentaire
```

```
>>> Age = Age + 1      # en plus court : Age += 1
```

```
>>> print (Age)
18
>>> Age = Age - 3      # en plus court : Age -= 3
>>> print (Age)
15
>>> Age = Age * 2      # en plus court : Age *= 2
>>> print (Age)
30
>>> a = 6*3-20
>>> print (a)
-2
>>> b = 25
>>> c = a + 2*b
>>> print (b,c)        # ne pas oublier la virgule
25 48
```

L'opérateur // donne la division entière :

```
>>> d = 450//360
>>> print (d)
1
```

L'opérateur % donne le reste de la division (opération modulo) :

```
>>> reste = 450 % 360
>>> print (reste)
90
```

L'opérateur ** donne la puissance :

```
>>> Mo = 2**20
>>> print (Mo)
```

2- Le type float (nombres en virgule flottante)

```
>>> b = 17.0    # le séparateur décimal est un point (et non une virgule)

>>> print (b)

17.0

>>> print (type(b))

<type 'float'>

>>> c = 14.0/3.0

>>> print (c)

4.66666666667

>>> c = 14.0//3.0    # division entière

>>> print (c)

4.0
```

Attention : avec des nombres entiers, l'opérateur / fait aussi une division entière :

```
>>> c = 14/3

>>> print (c)

4
```

Notation scientifique :

```
>>> a = -1.784892e4

>>> print (a)

-17848.92
```

3- Le type str (string : chaîne de caractères).

```
>>> Nom = 'Dupont'    # entre apostrophes

>>> print (Nom)

Dupont

>>> print (type(Nom))
```

```
<type 'str'>
```

```
>>> Prenom = "Pierre"          # on peut aussi utiliser les guillemets
```

```
>>> print (Prenom)
```

```
Pierre
```

```
>>> print (Nom, Prenom)        # ne pas oublier la virgule
```

```
Dupont Pierre
```

La concaténation désigne la mise bout à bout de plusieurs chaînes de caractères. La concaténation utilise l'opérateur +

```
>>> chaine = Nom + Prenom      # concaténation de deux chaînes de caractères
```

```
>>> print (chaine)
```

```
DupontPierre
```

```
>>> chaine = Prenom + Nom      # concaténation de deux chaînes de caractères
```

```
>>> print (chaine)
```

```
PierreDupont
```

```
>>> chaine = Prenom + ' ' + Nom
```

```
>>> print (chaine)
```

```
Pierre Dupont
```

```
>>> chaine = chaine + ' 18 ans'          # en plus court : chaine += ' 18 ans'
```

```
>>> print (chaine)
```

```
Pierre Dupont 18 ans
```

La fonction len() retourne la longueur (length) de la chaîne de caractères :

```
>>> print (len(chaine))
```

```
20
```

```
>>> print (chaine[0]) # premier caractère (indice 0)
```

```
P
```

```
>>> print (chaine[1]) # deuxième caractère (indice 1)
```

```

i
>>> print (chaine[1:4])
ier
>>> print (chaine[2:])
erre Dupont 18 ans
>>> print (chaine[-1]) # dernier caractère (indice -1)
s
>>> print (chaine[-6:])
18 ans
>>> chaine = 'Aujourd'hui'
SyntaxError: invalid syntax
>>> chaine = 'Aujourd\'hui' # séquence d'échappement \'
>>> print (chaine)
Aujourd'hui
>>> chaine = "Aujourd'hui"
>>> print (chaine)
Aujourd'hui

```

La séquence d'échappement `\n` représente un saut ligne :

```

>>> chaine = 'Première ligne\nDeuxième ligne'
>>> print (chaine)
Première ligne
Deuxième ligne

```

Plus simplement, on peut utiliser les triples guillemets (ou les triples apostrophes) pour encadrer une chaîne définie sur plusieurs lignes :

```

>>> chaine = """Première ligne
Deuxième ligne"""
>>> print (chaine)

```

Première ligne

Deuxième ligne

On ne peut pas mélanger les serviettes et les torchons (ici type str et type int) :

```
>>> chaine = '17.45'

>>> print (type(chaine))

<type 'str'>

>>> chaine = chaine + 2

TypeError: cannot concatenate 'str' and 'int' objects
```

La fonction float() permet de convertir un type str en type float

```
>>> nombre = float(chaine)

>>> print (nombre)

17.45

>>> print (type(nombre))

<type 'float'>

>>> nombre = nombre + 2                # en plus court : nombre += 2

>>> print (nombre)

19.45
```

La fonction input() lance une invite de commande (en anglais : prompt) pour saisir une chaîne de caractères.

```
>>> # saisir une chaîne de caractères et valider avec la touche Enter

>>> chaine = input("Entrer un nombre : ")

Entrer un nombre : 14.56

>>> print (chaine)

14.56

>>> print (type(chaine))

<type 'str'>
```

```
>>> nombre = float(chaine)      # conversion de type
>>> print (nombre**2)
211.9936
```